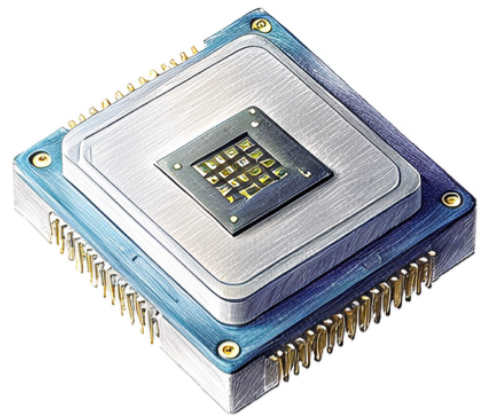


PROCESSOR ARCHITECTURE

Modern microprocessors are amongst the most complex systems ever created.



The instructions supported by a particular processor and their byte-level encodings are known as its ISA.

The ISA provides a conceptual layer of abstraction between compiler writers and chip designers.

NOTE - The way a processor operates can be different from the model of computation implied by the ISA. For example, the ISA can imply sequential execution of all operations in a program. However, a modern processor can execute multiple instructions parallelly to improve performance. Special mechanisms are used to ensure consistency in end results.

↳ This chapter uses a Y86-64 ISA defined below.

* Defining an ISA -

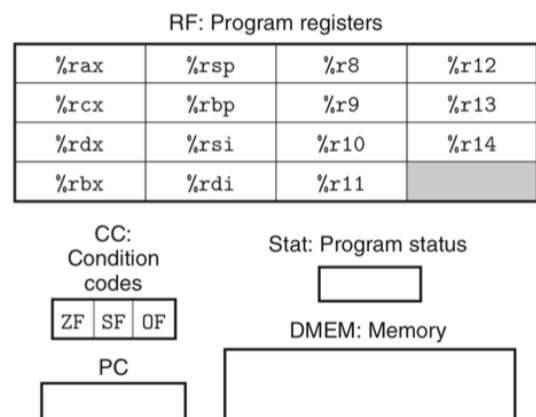
↳ This involves defining the various components of its state, the set of instructions and their encodings, a set of programming conventions and the handling of exceptional events.

Each instruction in a Y86-64 program can read and modify some part of the processor state → This is called programmer visible state.

↳ assembly code writer or the compiler

The state of Y86-64 is similar to x86-64 -

1. There are 15 program registers.
2. Each register stores a 64 bit word.
3. %rsp is used as the stack pointer.
4. There are three single bit condition codes.
5. The PC holds the currently executed instruction.
6. Memory is a large array of bytes holding program & data
↳ Y86-64 programs use virtual addresses.
7. The status code STAT indicates the overall status of execution.



The Y86-64 instruction set -

1. The move instructions - `movq`
 - ↳ `s` → represents the source. (Can be immediate (`i`), register (`r`) or memory (`m`))
 - ↳ `d` → represents the destination. (Can be register (`r`) or memory (`m`))
 - ↳ memory to memory transfers are disallowed like x86.
2. Integer Operations - `addq`, `subq`, `andq`, `xorq`
 - ↳ Only operate on register data
 - ↳ x86 allows integer operations directly on memory.

3. There are seven jump instructions, and six conditional move instructions.
4. Operations associated with the stack -
 - ↳ call ↳ pushq
 - ↳ ret ↳ popq
5. The halt instruction → stops the processor and sets status code to HLT
 - ↳ x86 programs have a similar operation but aren't permitted to use it.
 - ↳ causes the entire system to suspend.

Instruction Encoding - * **Instruction encoding needs to be unique.**

- Each instruction requires between 1 and 10 bytes.
- The first byte identifies the instruction type



The function values are relevant only when a group of related instructions share a common code.

- The 15 program registers have associated register IDs (0x00 to 0x0E)
 - ↳ The numbering matches x86-64.
 - ↳ Stored within the CPU in a register file.
 - ↳ Small random access file where register ID's serve as addresses.
 - ↳ The address 0x0F is used to indicate that no register is to be used.
- Instructions which require operands have a register specifier byte
 - ↳ Type | ID1 | ID2
 - ↳ Instructions with 1 operand have register ID2 set to 0x0F.
 - ↳ Instructions with no operands have no register specifier byte.
- Some instructions have an additional 8 byte word for immediate values.
 - ↳ Y86 uses absolute addresses as opposed to PC relative like x86.
 - ↳ Makes processor design a lot simpler.
 - ↳ All integers have little endian encoding, just as x86.

Exceptions -

- The programmer visible state includes a status code describing the overall status of the executing program.
- We use simple error handling → stop instruction execution on non-OK status.
- In x86, there are dedicated exception handlers which can trap into the OS.

* RISC vs CISC -

→ Complete set of differences can be found in the CS:APP textbook.

CISC -

- Large number of instructions.
- Some instructions have long execution times → Memory copies
- Variable sized instruction encodings.
- Multiple formats for operand specification.

RISC -

- Fewer instructions - typically < 100
- No instruction with long execution times
- Fixed length encoding → 4 byte encodings.
- Simple addressing format.
 - ↳ often just base & displacement.

